

Mentoria de IA aplicada · **Aula 5 de 8**

Sistemas multiagênticos e MCP

Times de agentes e uma tomada universal para ferramentas

Hoje

Ao final, você consegue:

- reconhecer **pipeline, supervisor, hierárquico, handoff** e **gerador + revisor**
- dizer quando multiagente é **exagero**
- explicar **MCP**: host, cliente, servidor
- diferenciar **tools, resources e prompts**
- escrever e conectar um **servidor MCP local**

Roteiro (40 min)

1. Padrões multiagente
 2. Quando é exagero
 3. MCP
 4. Demo: o time do curador
- Depois: 15 min de discussão

Divida em vários agentes só quando as partes forem
independentes.

E ligue todos ao mundo por um protocolo padrão: o servidor
não precisa saber quem vai usá-lo.

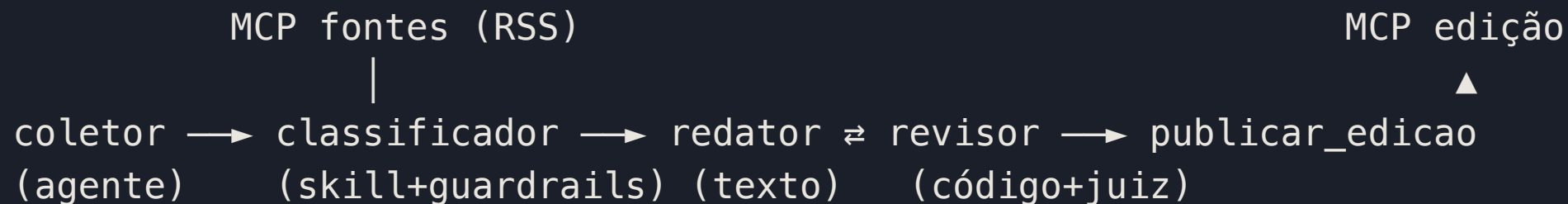
1 • Padrões multiagente

Como ligar os agentes

Cinco formas de organizar um time

Padrão	Como funciona	Exemplo
Pipeline	cada um faz uma etapa e passa adiante	coletor → classificador → redator → revisor
Supervisor	um agente central escolhe quem trabalha a cada rodada	pesquisador-chefe distribuindo subtarefas
Hierárquico	supervisores de supervisores	gerente com times de pesquisa e redação
Handoff	quem atende passa a conversa para outro	triagem → financeiro → técnico
Gerador + revisor	um gera, outro critica, até aprovar	redator ⇔ revisor

O time do curador



- Orquestração = **pipeline em código** (um workflow)
- Só o **coletor** é agente (escolhe as fontes)
- Redator $\rightleftharpoons$ revisor = **gerador + revisor**, até 2 tentativas

2 · Quando é exagero

Duas visões, um dia de diferença

Anthropic (13/jun/2025)

Agente-líder + subagentes **em paralelo** na pesquisa: **+90%** nas avaliações internas.

Mas **~15x mais tokens** que um chat. Ruim quando todos precisam do mesmo contexto, como em programação.

Cognition (12/jun/2025)

"Não construa multiagentes": em paralelo, sem o contexto um do outro, os agentes tomam **decisões conflitantes**.

Padrão: um agente, em sequência.

Ponto em comum: compensa quando as partes são **independentes** e podem rodar em paralelo.

3 • MCP

Model Context Protocol

Uma tomada universal

- **Host:** a aplicação com LLM (chat, editor, o nosso time)
- **Cliente:** o conector dentro do host, um por servidor
- **Servidor:** oferece ferramentas, dados e prompts

Mensagens em **JSON-RPC 2.0**.

Inspiração: o [Language Server Protocol](#) dos editores.

Transportes

stdio: subprocesso local; nada de `console.log` em `stdout`

Streamable HTTP: servidor remoto

O que um servidor oferece

	Quem decide usar	No projeto
Tools	o modelo	<code>listar_fontes</code> , <code>ler_fonte</code> , <code>publicar_edicao</code>
Resources	a aplicação	<code>noticia://{id}</code> , <code>edicao://ultima</code>
Prompts	o usuário	<code>classificar_noticia</code>

Anotações nas tools: `readOnlyHint`, `destructiveHint`, `idempotentHint`. São **dicas**, não garantias.

Ao vivo: o protocolo

demo:protocolo

```
→ tools/list
← { tools: [ { name: "buscar_termo", inputSchema: {...} }, ... ] }
→ tools/call { name: "buscar_termo", arguments: { termo: "blockchain" } }
← { content: [ "termo não encontrado: blockchain" ], isError: true }
```

Era 2025-11-25

initialize abre uma sessão. Padrão do cliente do SDK.

Era 2026-07-28

Sem sessão: cada requisição leva a versão no `_meta`.

Segurança

- **Menor privilégio:** o coletor só recebe `listar_fontes` e `ler_fonte`
- **Valide o que vem do modelo:** `ler_fonte("../../etc/passwd")` → **erro**, nenhum arquivo lido
- **Saída de servidor é dado não confiável:** pode trazer injeção (Aula 3)
- **Ferramenta que muda algo** → aprovação humana (Aula 7)
- Só instale servidores de **fontes confiáveis**

4 · O time do curador

demo:time

Resultado do time

Papel	Chamadas	Tempo
Coletor (agente, MCP)	1 · 12 passos	69 s
Classificador	32	97 s
Redator	14	60 s
Revisor (código + juiz)	14	616 s

- **Gargalo:** o juiz do revisor (73% do tempo), não o agente
- **9 de 10** aprovados; **n10** reprovada 2× ("reduz" × "mede menos")
- Edição **sem** regulação nem mercado: "top 10 por confiança" + empates

Discussão

15 minutos

Para conversar

1. O curador precisava de **quatro** agentes? O que o time ganhou em relação ao pipeline da Aula 4?
2. Um processo do seu time lembra qual padrão: pipeline, supervisor, handoff ou gerador + revisor?
3. Por que `buscar_termo` é tool e `glossario://{termo}` é resource, se devolvem a mesma informação?
4. Que servidor MCP você gostaria de ter no seu trabalho? Que tools ele teria e quais seriam só leitura?
5. O que pode dar errado se você instalar um servidor MCP de uma fonte desconhecida?

Para a próxima aula

Exercício

Um servidor MCP (glossário) e os padrões como funções

```
pnpm -F @mentoria/ex05-mcp test
```

Desafio: handoff com detecção de ping-pong

Projeto final · M5

Servidores de fontes e edição, adaptador e o time

```
pnpm -F @mentoria/curador m05:edicao
```

Traga a edição e o relatório

Próxima aula · **Orquestração com LangGraph.js**

E se o pipeline do time virasse um **grafo, com estado salvo e caminhos que dependem da confiança?**

Checkpoint para retomar de onde parou e arestas condicionais.